

Python Data Structures & LeetCode Foundations

A Quick-Reference Guide for Technical Interviews and Algorithm Design

1. BIG-O COMPLEXITY QUICK REFERENCE

In technical interviews, your initial solution must always be evaluated for time and space scalability. Memory allocation and runtime speeds are measured across asymptotic constraints:

Notation	Classification	Common LeetCode Context / Trigger Patterns
$O(1)$	Constant	Hash map lookups, array index access, mathematical equations, push/pop to stack.
$O(\log N)$	Logarithmic	Binary Search, searching or balancing elements in a binary search tree (BST), heaps.
$O(N)$	Linear	Single pass over an array/string, linear scan, sliding window, tree traversal (BFS/DFS).
$O(N \log N)$	Linearithmic	Sorting standard collections (Python's Timsort via <code>.sort()</code> or <code>sorted()</code>), heap sorts.
$O(N^2)$	Quadratic	Nested iterations over a collection, checking all unique pairs (brute-force arrays).
$O(2^N)$	Exponential	Generating combinations, raw combinatorial subsets, unoptimized recursive fibonacci.

2. CORE PYTHON DATA STRUCTURES

A. Lists (Dynamic Arrays)

Python lists are continuous blocks of memory allocated dynamically behind the scenes. Inserting or removing from the end is fast, but operations at the front require shuffling elements down the chain.

Operation Code	Time Complexity	Interview Mechanics / Notes
<code>arr.append(val)</code>	$O(1)$	Appends value to the right-hand end. Amortized constant time.
<code>arr.pop()</code>	$O(1)$	Removes and returns the rightmost element.
<code>arr.insert(0, val)</code>	$O(N)$	Worst-case. Shifts all elements rightward by 1 index. Avoid in loops.
<code>arr[i]</code>	$O(1)$	Direct address access via index lookup.

```
# Implementation Snippet
arr = [1, 2, 3]
arr.append(4)      # [1, 2, 3, 4]
last = arr.pop()   # last = 4, arr = [1, 2, 3]
```

B. Deque (Double-Ended Queues / Stacks / Queues)

When you need to implement a true **Queue (FIFO)** or a **Stack (LIFO)** with fast operations at both boundaries, utilize `collections.deque`. It uses a doubly-linked list infrastructure internally.

Operation Code	Time Complexity	Interview Mechanics / Notes
<code>q.append(val)</code>	$O(1)$	Pushes value onto the right side.
<code>q.appendleft(val)</code>	$O(1)$	Pushes value onto the front left boundary. Perfect for Queue management.
<code>q.popleft()</code>	$O(1)$	Removes and returns the leftmost item. Critical for BFS traversals .

```
from collections import deque
queue = deque([2, 3])
queue.append(4)          # deque([2, 3, 4])
first = queue.popleft()  # first = 2, queue = deque([3, 4])
```

C. Dictionaries & Sets (Hash Maps & Hash Sets)

The single most powerful tool in LeetCode algorithms. Python uses hash functions to map keys directly to internal index arrays, enabling near-instantaneous searches.

Operation Code	Time Complexity	Interview Mechanics / Notes
<code>d[key] = val</code>	$O(1)$	Inserts key-value mapping into hash bucket locations.
<code>if key in d:</code>	$O(1)$	Instant check for item existence. Avoids nested scans.
<code>s.add(val)</code>	$O(1)$	Inserts element into a set. Ensures all entries are unique.

```
# Frequency Counter Pattern
counts = {}
for char in "abbc":
    counts[char] = counts.get(char, 0) + 1  # {'a': 1, 'b': 2, 'c': 1}
```

D. Heaps (Priority Queues)

Python provides a min-heap structure natively via the `heapq` module. The smallest item always sits at index `0`.

- `heapq.heappush(heap, val)` : Inserts element, auto-balancing in **$O(\log N)$** .
- `heapq.heappop(heap)` : Extracts the minimum value in **$O(\log N)$** .
- **Max-Heap Trick:** Invert your numeric entries to negative values (e.g., store `-val`) when pushed, then invert back upon extraction.

3. ESSENTIAL LEETCODE PATTERNS & SCAFFOLDING

A. Two-Pointer Approach (Array Sort / Target Sum)

Used to find pairs or sections in a sorted array by moving pointers from both ends inward, minimizing duplicate iterations.

```
def two_sum_sorted(nums: list[int], target: int) -> list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1 # Need a larger number
        else:
            right -= 1 # Need a smaller number
    return []
```

B. Depth-First Search (DFS) on Binary Trees

Explores a path completely down to its leaf nodes recursively before backtracking to adjacent paths. Excellent for tree validations.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth(root: TreeNode) -> int:
    if not root:
        return 0
    left_side = max_depth(root.left)
    right_side = max_depth(root.right)
    return max(left_side, right_side) + 1
```

C. Breadth-First Search (BFS) on Graphs / Trees

Explores nodes level by level horizontally. Indispensable for tracking the **shortest path** across unweighted structures.

```
def level_order_traversal(root: TreeNode) -> list[list[int]]:
    if not root:
        return []
    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        current_level = []
        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(current_level)
    return result
```

💡 Professional Interview Coding Strategies

- **Clarify Constraints First:** Ask about input size boundaries (e.g., "Can N be zero? Are negative values possible?"). This signals your logic tracks edge cases natively.
- **State Complexities Before Writing:** Explicitly mention: *"I can solve this via a hash table tracking approach in $O(N)$ time and $O(N)$ auxiliary space."*
- **Dry-Run Code Manually:** Before declaring completion, walk your interviewer through an example input array using step-by-step index tracer tracking out loud.